

AI-DimiOJ 프로젝트 기획서 및 결과 보고서

AI-DimiOJ

AI 코딩 선생님이 함께하는
중고등학생용 온라인 코딩 학습 플랫폼

프로젝트 기획서 및 결과 보고서

학교 / 수업	디지털미디어고등학교 2학년 1학기 정보통신
학번 / 이름	20311 박진우
저장소	github.com/goodpjw2008/Project_AI-DimiOJ

제1부. 프로젝트 기획서

1. 프로젝트 기본 내용

프로젝트 제목

AI-DimiOJ (Dimigo Online Judge) - 중고등학생을 위한 온라인 코딩 문제 해결 플랫폼

프로젝트 정보

- 학교/수업: 디지털미디어고등학교 2학년 1학기 정보통신
- 학번 / 이름: 20311 박진우

2. 기획 배경 및 목적

주제 선정 이유

- 기존 온라인 저지 시스템(BOJ, Codeforces 등)은 초보자에게 진입장벽이 높음
- 중고등학생들이 프로그래밍을 학습할 때 필요한 단계별 학습 환경 부족
- 학교 내에서 코딩 교육과 평가를 위한 전용 플랫폼의 필요성
- AI 기술을 활용한 개인화된 학습 지원 시스템의 부재

목표 사용자

중고등학생 코딩 초보자

- 프로그래밍을 처음 배우는 학생들
- 알고리즘 문제 해결 능력을 기르고 싶은 학생들
- 코딩 테스트 준비를 하는 학생들
- 학교 정보 수업을 듣는 학생들

예상 기대효과

- 학습 동기 부여: 랭킹 시스템과 문제 해결 현황을 통한 성취감 제공
- 단계적 학습: 난이도별 문제 제공으로 체계적인 실력 향상
- 즉시 피드백: 실시간 채점 시스템으로 빠른 학습 효과
- 커뮤니티 형성: 게시판을 통한 학습자 간 정보 공유 및 협력

- **개인화 학습:** AI 챗봇을 통한 맞춤형 힌트 및 설명 제공
-

3. 주요 기능 및 기술

필수 기능 목록

1. 사용자 관리

- 회원가입/로그인 (암호화된 사용자 정보 관리)
- 프로필 관리 (외부 플랫폼 연동: Codeforces, AtCoder)
- 비밀번호 보안 규칙 적용

2. 문제 관리

- 문제 등록/수정/삭제
- 테스트케이스 관리
- 문제 난이도 및 카테고리 분류
- 예제 입출력 제공

3. 채점 시스템

- 다중 언어 지원 (Python, C, C++)
- 실시간 코드 실행 및 채점
- 시간/메모리 제한 관리
- 커스텀 체커 지원

4. 제출 관리

- 코드 제출 및 결과 확인
- 제출 기록 조회
- 코드 보기 기능

5. 랭킹 시스템

- 해결한 문제 수 기준 순위
- 외부 플랫폼 티어 표시

추가 기능

1. AI 챗봇

- OpenAI API를 활용한 문제 해결 도움
- 실시간 스트리밍 응답
- 문제별 맞춤형 힌트 제공

2. 커뮤니티

- 게시판 (질문/답변, 공지사항)
- 게시물 작성/수정/삭제

3. 보안 기능

- 악성 요청 차단
- 속도 제한 (Rate Limiting)

사용할 기술 또는 도구

- **Backend:** Python Flask, SQLAlchemy
 - **Database:** SQLite (개발), PostgreSQL/MySQL (운영)
 - **Frontend:** HTML5, CSS3, JavaScript, Bootstrap
 - **Security:** AES-256 암호화, PBKDF2 해싱
 - **AI:** OpenAI GPT API
 - **Deployment:** Linux 서버
 - **Version Control:** Git
-

4. 페이지별 핵심 내용 요약

메인 화면 (/)

- 플랫폼 소개 및 주요 기능 안내
- 로그인/회원가입 링크
- 최신 공지사항 표시

문제 목록 (/problems)

- 페이지네이션이 적용된 문제 리스트
- 문제별 해결 상태 표시 (미시도/시도/해결)
- 정답률 및 제출 통계 표시

문제 상세 (/problems/<id>)

- 문제 설명, 입출력 명세, 예제
- 코드 에디터 (다중 언어 지원)
- AI 챗봇 연동 (문제 해결 도움)
- 제출 기록 및 결과 확인

제출 현황 (`/submissions` , `/my_submissions`)

- 전체/개인 제출 기록 조회
- 제출 결과 및 실행 시간/메모리 표시
- 코드 보기 기능

랭킹 (`/ranking`)

- 해결한 문제 수 기준 사용자 순위
- 외부 플랫폼 티어 표시
- 프로필 링크 연동

프로필 (`/profile/<nickname>`)

- 개인 통계 (총 제출, 해결 문제 수, 정답률)
- 외부 플랫폼 연동 정보
- 프로필 수정 기능

게시판 (`/forum`)

- 질문/답변 게시판
- 게시글 작성/수정/삭제
- 페이지네이션 적용

관리 기능

- 문제 생성/수정 (`/create` , `/problems/<id>/edit`)
- 테스트케이스 관리 (`/problems/<id>/testcases`)

5. 디자인(UI/UX) 계획

전체 스타일 컨셉

- 미니멀하고 깔끔한 디자인: 학습에 집중할 수 있는 심플한 인터페이스
- 직관적인 네비게이션: 초보자도 쉽게 사용할 수 있는 구조
- 반응형 디자인: 다양한 디바이스에서 최적화된 경험

주요 색상 및 폰트

- Primary Color: #333 (네비게이션 바)

- **Background:** #f9f9f9 (연한 회색)
- **Content Area:** #fff (흰색)
- **Accent Colors:**
 - 성공: #d4edda (연한 초록)
 - 경고: #fff3cd (연한 노랑)
 - 오류: #f8d7da (연한 빨강)
- **Font:** Arial, sans-serif (가독성 우선)

참고 사이트

- **백준 온라인 저지:** 문제 목록 및 제출 현황 UI 참고
 - **Codeforces:** 랭킹 시스템 및 프로필 페이지 참고
 - **LeetCode:** 코드 에디터 및 문제 상세 페이지 참고
 - **GitHub:** 깔끔한 테이블 디자인 및 페이지네이션 참고
-

6. 개발 일정 및 작업 계획

1주차: 프로젝트 설계 및 환경 구축

- 프로젝트 기획서 작성 및 요구사항 분석
- 개발 환경 설정 (Flask, 데이터베이스)
- 기본 프로젝트 구조 설계
- Git 저장소 설정 및 초기 커밋

2주차: 사용자 관리 시스템 구현

- 데이터베이스 모델 설계 (User, Problem, Submission 등)
- 회원가입/로그인 기능 구현
- 사용자 인증 및 세션 관리
- 비밀번호 암호화 및 보안 기능

3주차: 문제 관리 시스템 구현

- 문제 등록/수정/삭제 기능
- 테스트케이스 관리 시스템
- 문제 목록 페이지 (페이지네이션 포함)
- 문제 상세 페이지 기본 구조

4주차: 채점 시스템 구현

- 코드 실행 환경 구축 (샌드박스)
- 다중 언어 지원 (Python, C, C++)
- 시간/메모리 제한 관리
- 채점 결과 처리 및 저장

5주차: 제출 관리 및 랭킹 시스템

- 코드 제출 기능 완성
- 제출 기록 조회 페이지
- 랭킹 시스템 구현
- 프로필 페이지 및 통계 기능

6주차: AI 챗봇 및 추가 기능

- OpenAI API 연동
- 실시간 스트리밍 챗봇 구현
- 게시판 기능 구현
- 외부 플랫폼 연동 (Codeforces, AtCoder)

7주차: 보안 및 성능 최적화

- 보안 미들웨어 구현 (악성 요청 차단)
- 속도 제한 (Rate Limiting) 적용
- 성능 최적화 및 버그 수정

8주차: UI/UX 개선 및 테스트

- 프론트엔드 디자인 완성
- 반응형 디자인 적용
- 사용자 테스트 및 피드백 수집
- 최종 버그 수정 및 안정화

9주차: 발표 준비 및 배포

- 프로젝트 문서화 완성
 - 발표 자료 준비
 - 최종 테스트 및 배포
 - 프로젝트 발표
-

추가 고려사항

확장 가능성

- 문제 카테고리 및 태그 시스템
- 대회 기능 (Contest Mode)
- 팀 기능 및 협업 도구
- 모바일 앱 개발

운영 및 유지보수

- 로그 시스템 및 모니터링
- 백업 및 복구 시스템
- 사용자 피드백 수집 체계
- 지속적인 문제 추가 및 관리

제2부. 프로젝트 결과 보고

중고등학생을 위한 온라인 코딩 문제 해결 플랫폼. 알고리즘 문제 풀이, 자동 채점, AI 코딩 선생님(GPT) 연동, 게시판, 랭킹을 제공합니다.



프로젝트 정보 본 프로젝트는 디지털미디어고등학교 2학년 1학기 정보통신 수업의 교과 프로젝트로 진행되었습니다.

주요 기능

- **사용자 관리:** 회원가입/로그인, AES 암호화된 사용자명, PBKDF2 해시 비밀번호
- **문제 관리:** 문제·테스트케이스 등록, 제한시간/메모리 설정
- **자동 채점:** Python / C / C++ 다중 언어, 샌드박스 실행, 시간·메모리 제한
- **제출 기록:** 개인/전체 제출 현황, 결과 및 코드 확인
- **랭킹:** 해결한 문제 수 기준, Codeforces/AtCoder 티어 연동
- **게시판:** 질문/답변 게시판 (작성/수정/삭제, 페이지네이션)
- **AI 코딩 선생님:** OpenAI `gpt-4o-mini` 기반 문제별 힌트 및 스트리밍 응답
- **보안:** 악성 요청 차단, Rate Limiting (IP당 10초/20회)

사용 방법



단계별 상세 설명, 화면 설명, FAQ까지 포함된 **사용자 가이드 (USER_GUIDE.md)** 를 함께 참고해 주세요.

1) 문제 목록 둘러보기

상단 네비게이션의 **문제 목록**에서 풀고 싶은 문제를 선택합니다. 각 문제의 제출 수, 정답 수, 정답률을 한눈에 확인할 수 있습니다.

Dimigo Online Judge

goodpjw2008 | 로그아웃

문제 목록

제출 현황

내 제출

게시판

랭킹

문제 제작

문제 목록

번호	문제 이름	제출 수	정답 수	정답률
1	(입출력과 사칙연산)A+B	0	0	0%
2	(입출력과 사칙연산)A-B	0	0	0%
3	(입출력과 사칙연산)A×B	0	0	0%
4	(입출력과 사칙연산)A/B	0	0	0%
5	사칙연산	0	0	0%
6	(조건문)두 수 비교하기	0	0	0%
7	(조건문)시험 성적	0	0	0%
8	(조건문)윤년	0	0	0%
9	(반복문)구구단	0	0	0%
10	(반복문)합	0	0	0%
11	(반복문)영수증	1	1	100.0%
12	(반복문)별 찍기	0	0	0%
13	(배열)개수 세기	0	0	0%
14	(배열)X보다 작은 수	0	0	0%

처음

이전

1

2

3

다음

마지막

2) AI 디미 코딩 선생님에게 질문하기

문제 페이지 우측의 **AI 디미 코딩 선생님** 채팅창에서 모르는 개념을 물어보면, GPT 기반 챗봇이 친절하게 설명하고 필요할 때 코드 예시도 제공합니다. (질문은 오른쪽 파란 말풍선, AI 답변은 왼쪽 회색 말풍선으로 구분됩니다.)

Dimigo Online Judge

goodpjw2008 | 로그아웃

문제 목록

제출 현황

내 제출

게시판

랭킹

문제 제작

(반복문)별 찍기

문제 수정

제출

내 제출

채점 현황

테스트케이스 관리

문제 삭제

- 시간 제한: 1초
- 메모리 제한: 128MB
- 제출: 0
- 정답: 0
- 정답률: 0%
- 테스트케이스 수: 3

지문

첫째 줄에는 별 1개, 둘째 줄에는 별 2개, N번째 줄에는 별 N개를 찍는 문제

입력

첫째 줄에 $N(1 \leq N \leq 100)$ 이 주어진다.

출력

첫째 줄부터 N번째 줄까지 차례대로 별을 출력한다.

예제 입출력

Sample Input:

```
5
```

Sample Output:

```
*
**
***
****
*****
```

AI 디미 코딩 선생님

대화내용 삭제

배열이 뭐니?

배열은 여러 개의 데이터를 하나의 변수에 묶어서 관리할 수 있는 데이터 구조예요. 맥락에 따라 설명하면, 배열은 같은 데이터 타입의 여러 값을 저장할 수 있는 선형적 구조입니다.

예를 들어, 여러 개의 숫자를 저장하기 위해 배열을 사용할 수 있어요. 배열의 각 요소는 인덱스(위치 번호)를 가지고 있어서 쉽게 접근할 수 있습니다. 다루는 데이터가 많을 때, 배열을 사용하면 더 효율적이고 관리하기 쉬워요.

앞서 주신 별 찍기 문제와 관련하여, 배열보다는 반복문을 사용해 문제를 해결하는 것이 일반적이지만, 배열을 쓰는 예시를 보여줄 수 있어요. 예를 들어, N개의 별을 행마다 저장해두고 출력하는 방법은 이런 형태가 될 수 있어요:

Code

Copy

```
n = 5 # 예제 입력: N에 5를 대입
stars = [] # 빈 배열 생성

for i in range(1, n + 1): # 1부터 N까지 반복
```

AI 디미 코딩 선생님에게 질문해보세요!

전송

후속 질문도 자유롭게 이어갈 수 있습니다. 이전 대화 맥락을 기억해 답변하므로 단계적인 학습이 가능합니다.

Dimigo Online Judge

goodpiw2008 | 로그인

문제 목록 제출 현황 내 제출 게시판 랭킹 문제 제작

(자료구조)트리의 부모 찾기

문제 수정

제출

내 제출

재점 현황

테스트케이스 관리

문제 삭제

시간 제한: 1초

메모리 제한: 128MB

제출: 0

정답: 0

정답률: 0%

테스트케이스 수: 3

지문

루트 없는 트리가 주어진다. 이때, 트리의 루트를 1이라고 정했을 때, 각 노드의 부모를 구하는 프로그램을 작성하시오.

입력

첫째 줄에 노드의 개수 N ($2 \leq N \leq 100,000$)이 주어진다. 둘째 줄부터 $N-1$ 개의 줄에 트리 상에서 연결된 두 정점이 주어진다.

출력

첫째 줄부터 $N-1$ 개의 줄에 각 노드의 부모 노드 번호를 2번 노드부터 순서대로 출력한다.

예제 입력

Sample Input:

7
1 6
6 3
3 5
4 1
2 4
4 7

AI 디미 코딩 선생님

대화내용 삭제

이 문제는 너무 어렵다. 쉽게 문제에 대해서 알려줘. 답은 알려주면 안돼.

이 문제는 트리는 자료구조의 한 종류를 다루고 있어요. 트리는 노드들이 연결된 구조로, 부모-자식 관계가 존재하는데, 주어진 문제에서는 각 노드의 부모 노드를 찾는 것이 목표예요.

트리의 기본 개념.

- 트리는 하나의 루트 노드에서 시작되어 모든 노드가 연결된 계층 구조입니다.

- 노드는 각각의 데이터 블록을 나타내며, 부모 노드는 각 노드를 하나의 상위 노드로 연결합니다.

- 루트 노드는 부모가 없고, 나머지 노드들은 하나의 부모를 가지고 있어요.

주어진 문제의 요점.

1. 입력 처리: 노드의 개수를 입력 받아요. 그런 다음, 연결된 두 노드의 정보(N-1줄)를 받아서 트리를 구성하는데 사용해요.

2. 트리 구성: 각 노드에 대한 연결 정보를 통해 트리를 만들고, 그 구조를 기반으로 각 노드의 부모를 찾아야 해요.

3. 출력: 최종적으로 각 노드(2부터 N까지)의 부모 노드 번호를 출력해야 해요.

문제 해결 방법.

- 깊이 우선 탐색(DFS)이나 너비 우선 탐색(BFS)을 이용하여 각 노드를 탐색하면서 부모 노드를 기록하는 방식으로 접근할 수 있어요.

- 연결 정보를 기반으로 트리를 순회하면서 각 노드에 대한 부모 정보를 찾는 것이 중요해요.

이런 식으로 문제를 접근해보면, 해결의 실마리를 찾을 수 있을 거예요.

AI 디미 코딩 선생님께 질문해보세요!

전송

3) 코드 작성 후 제출하기

문제 페이지 하단의 **제출창**에서 언어(Python / C / C++)를 선택하고 코드를 작성한 뒤 **제출** 버튼을 누릅니다. 샌드박스에서 자동으로 컴파일·실행되어 채점됩니다.

출력

첫째 줄부터 N개의 줄에 오름차순으로 정렬한 결과를 한 줄에 하나씩 출력한다.

예제 입력

Sample Input:

5
5
2
3
4
1

Sample Output:

1
2
3
4
5

제출창

Python

입력받은 수를 저장할 리스트
numbers = []

수의 개수 N을 입력받음
N = int(input())

N개의 수를 입력받아 리스트에 추가
for _ in range(N):
 number = int(input())
 numbers.append(number)

제출

나의 제출

아직 제출 내역이 없습니다.

이 문제를 요약해줘

이 문제는 주어진 N개의 정수를 오름차순으로 정렬하여 출력하는 프로그램을 작성하는 것이다.

요약:

- 문제 제목: 수 정렬하기

- 문제 설명: N개의 숫자를 오름차순으로 정렬하는 프로그램을 작성

- 입력:

- 첫째 줄: 수의 개수 N ($1 \leq N \leq 1,000$)

- 둘째 줄부터 N개의 줄: 정수 (정값이 1,000 이하이며 중복되지 않음)

- 출력:

- 오름차순으로 정렬된 숫자를 한 줄에 하나씩 출력

- 예제:

- 입력:

5
5
2
3
4
1

AI 디미 코딩 선생님께 질문해보세요!

전송

4) 제출 결과 확인하기

상단 **내 제출** 메뉴에서 제출 기록과 결과(AC/WA/CE/TLE/RE), 실행 시간, 메모리 사용량, 코드 길이를 확인할 수 있습니다. **보기** 링크로 제출한 코드 원본도 다시 볼 수 있습니다.

Dimigo Online Judge
goodpiw2008 | 로그아웃

[문제 목록](#)
[제출 현황](#)
[내 제출](#)
[게시판](#)
[랭킹](#)
[문제 제작](#)

내 제출 기록

ID	문제	언어	결과	시간(s)	메모리(MB)	코드 길이	코드 보기	제출 시간
8	25	py	AC	0.000	-	273	보기	2026-05-21 02:16:05.391168
7	12	py	WA	0.002	22656	194	보기	2026-05-21 02:14:34.673581
6	12	py	WA	0.002	22944	194	보기	2026-05-21 02:14:34.670683
5	12	cpp	CE	0.000	-	194	보기	2026-05-21 02:14:34.575882
4	12	cpp	CE	0.000	-	194	보기	2026-05-21 02:14:34.575714
3	28	py	AC	0.002	10080	800	보기	2026-05-21 02:08:21.498938
2	24	py	AC	0.002	10080	850	보기	2026-05-21 02:07:32.616460
1	11	py	AC	0.003	10080	416	보기	2026-05-21 02:06:17.972797

5) 직접 문제 출제하기

상단 **문제 제작** 메뉴에서 제목·지문·입력/출력 설명·시간/메모리 제한을 입력해 새 문제를 만들 수 있습니다. 생성 후 테스트케이스를 추가하면 다른 사용자가 풀 수 있는 문제가 됩니다.

Dimigo Online Judge
goodpiw2008 | 로그아웃

[문제 목록](#)
[제출 현황](#)
[내 제출](#)
[게시판](#)
[랭킹](#)
[문제 제작](#)

문제 제작

제목

지문 (description)

입력 설명 (input specification)

출력 설명 (output specification)

시간 제한(초)

1

메모리 제한(MB)

128

설치

```
# 1) 저장소 클론
git clone <YOUR_REPO_URL> dimeoj
cd dimeoj

# 2) 가상환경 생성 및 활성화
python3 -m venv myenv
source myenv/bin/activate          # Windows: myenv\Scripts\activate

# 3) 의존성 설치
pip install -r requirements.txt

# 4) 환경 변수 설정
cp .env.example .env
# .env 파일을 열어 OPENAI_API_KEY 등을 본인 값으로 채워 넣기

# 5) DB 마이그레이션
flask db upgrade

# 6) (선택) 샘플 문제 시딩
python init_problems_with_testcases.py
```

실행

manage.sh 스크립트로 서버를 관리합니다.

```
./manage.sh start      # 서버 시작 (기본 0.0.0.0:5001)
./manage.sh stop       # 서버 종료
./manage.sh restart    # 재시작
./manage.sh status     # 실행 상태 확인
```

환경 변수로 호스트/포트/워커 수를 조정할 수 있습니다.

```
PORT=8080 WORKERS=4 ./manage.sh start
```

- PID 파일: **server.pid**
- 로그 파일: **server.log**

개발 시에는 Flask 개발 서버도 사용 가능합니다.

```
python app.py
```

디렉토리 구조

```
.
├── app.py                # Flask 라우트 및 메인 진입점
├── models.py             # SQLAlchemy 모델 (User, Problem, Submission,
│   └── Post, TestCase)
├── judge.py              # 채점 엔진 (샌드박스 실행, 결과 판정)
├── utils.py              # 암호화/해시/외부 API 헬퍼
├── config.py             # 환경 설정 (.env 로드)
├── manage.sh             # 서버 라이프사이클 관리 스크립트
├── run_with_gunicorn.py  # Gunicorn 직접 실행 스크립트
├── requirements.txt
├── .env.example          # 환경 변수 템플릿
├── init_problems.py      # 문제 시딩 스크립트
├── init_problems_with_testcases.py # 문제 + 테스트케이스 시딩
├── migrations/           # Flask-Migrate 마이그레이션 히스토리
├── instance/             # SQLite DB 파일 (gitignore)
├── sandbox/              # 채점 시 코드 실행 격리 디렉토리 (gitignore)
├── static/               # CSS, 이미지
├── templates/            # Jinja2 템플릿
└── images/              # README용 스크린샷
```

환경 변수 (`.env`)

키	설명	기본값
<code>OPENAI_API_KEY</code>	AI 코딩 선생님용 OpenAI API 키	(필수)

 `.env` 파일은 `.gitignore` 에 등록되어 있어 커밋되지 않습니다. 절대 API 키를 저장소에 올리지 마세요.

라이선스

[MIT License](#) 하에 배포됩니다. Copyright © 2026 goodpjw2008.

사용된 오픈소스 라이브러리 및 라이선스 목록은 [DEPENDENCIES.md](#)를 참고하세요.

본 프로젝트는 디지털미디어고등학교 2학년 1학기 **정보통신 수업** 교과 프로젝트로 제작되었습니다.